

1 Relatório de auditoria técnica — SISPNAIST

Revisão realizada em: 1º de outubro de 2026

Projeto avaliado: frontend React/TypeScript/Vite na Vercel, API Node.js/Express/TypeScript no Render e MongoDB Atlas

Classificação geral: **atenção alta** — corrigir primeiro credenciais e autorização/isolamento de dados

1.1 Resumo executivo

Revisei o arquivo de projeto recebido, os fluxos de autenticação, as rotas de dados de trabalhadores e saúde, o isolamento por empresa, uploads, auditoria, configuração de deploy, dependências e testes. Também verifiquei as URLs públicas e rodei os comandos de build/teste/auditoria no código extraído. **Não alterei a aplicação, as contas de Vercel/Render/Atlas, nem dados em produção.**

Há proteções bem encaminhadas: backend TypeScript compila; os 129 testes do backend passaram; há autenticação em várias áreas, schemas Joi, hashing de senhas, CSRF e tratamento privado de arquivos. Porém, encontrei riscos que devem ser corrigidos antes de confiar o sistema a dados reais de saúde/identificação: variáveis com credenciais preenchidas vieram dentro do arquivo compactado; algumas rotas de escrita usam apenas autenticação e não confirmam se o trabalhador-alvo pertence ao escopo da pessoa logada; um gestor sem empresa associada pode gerar consulta sem filtro; a expiração configurada do token de acesso é de sete dias; e o token de verificação de e-mail pode sair na resposta de cadastro.

A auditoria de dependências também precisa de ação: `npm audit` identificou **17 advisories no backend** (12 altos, 4 moderados e 1 baixo; 7 no recorte de produção) e **22 no frontend** (14 altos, 6 moderados e 2 baixos). Além disso, a versão de Multer no backend é 1.4.5-lts.2, anterior à versão corrigida indicada pelo Express/OpenJS para avisos de segurança de agosto de 2026.

1.2 Escopo e limites

A revisão é uma análise estática do pacote enviado, com leitura aprofundada das rotas, controllers, autorização, modelos e configurações de maior risco, complementada por inventário de arquivos e verificações automatizadas. Não tive acesso autenticado a Vercel, Render ou Atlas, não executei fluxos com contas reais e não fiz teste de intrusão nem consulta a documentos pessoais em produção. As condições manuais no painel do Render/Atlas — por exemplo, variáveis de ambiente, backup e allowlist de rede — precisam ser verificadas pelo responsável.

As respostas HTTP foram somente verificações de disponibilidade: a home da Vercel e os endpoints `/health` e `/api/health` do Render responderam 200. Isso **não** prova que o banco esteja pronto, que o envio de e-mail funcione ou que as permissões dos dados estejam corretas.

1.3 Resultados de validação

Verificação	Resultado observado
Backend <code>npm run build</code>	Passou
Backend <code>npm test -- --runInBand -- forceExit</code>	9 suítes e 129 testes passaram; Jest emitiu aviso de handles abertos por causa do <code>--forceExit</code>
Frontend <code>npm run build</code>	Passou com Vite

Verificação	Resultado observado
Frontend <code>npm run type-check</code>	Falhou: 109 erros TypeScript em 28 arquivos
<code>npm run lint</code> no backend e no frontend	Falhou: não há configuração ESLint que permita executar o script
<code>npm audit backend</code>	17 avisos: 12 altos, 4 moderados, 1 baixo; --omit=dev: 7 avisos, incluindo 3 altos
<code>npm audit frontend</code>	22 avisos: 14 altos, 6 moderados, 2 baixos
URLs públicas	Home Vercel e health Render responderam HTTP 200

A diferença entre a build Vite e o type-check é importante: o script build do frontend executa vite build sem rodar tsc. Assim, a build pode sair mesmo com os 109 erros de tipos. O workflow GitHub, por outro lado, executa npm run type-check; como os deploys dependem desse job, os erros atuais tendem a impedir os deploys pelo workflow. Isso é uma inferência baseada no YAML e nos checks locais, não uma leitura do histórico de execuções do GitHub.

1.4 Achados e mudanças recomendadas

1.4.1 A0 — Credenciais presentes no pacote enviado

Prioridade: imediata, se os valores forem ativos. O pacote contém backend/.env com valores preenchidos para variáveis que incluem MONGODB_URI, JWT_SECRET, JWT_REFRESH_SECRET, credenciais de e-mail, BREVO_API_KEY e REDIS_URL. Há também um backend/admin.txt curto e não vazio cuja finalidade não é identificável pelo nome/conteúdo estrutural. Não reproduzi nem registrei os valores no relatório.

Por que importa: se forem credenciais reais, qualquer pessoa com acesso ao arquivo pode tentar usar o banco, assinar tokens, operar o provedor de e-mail ou acessar Redis. O fato de .env estar no .gitignore não remove o valor de um arquivo já anexado/compartilhado, nem garante que ele nunca tenha sido commitado.

Ação: trate os valores ativos como expostos. Revogue e gere novas credenciais para Atlas, JWT de acesso, JWT de refresh, e-mail e Redis; invalide sessões existentes; confira os logs e a allowlist do Atlas; remova os arquivos de pacotes compartilháveis e confira o histórico Git. Use apenas variáveis do painel de deploy/secret manager. Não cole segredos novos em prompts ou chats. Se os valores forem exemplos sem validade, registre isso e mantenha o arquivo fora dos pacotes.

1.4.2 A1 — Escritas de registros de saúde sem prova consistente de escopo

Prioridade: alta. Em backend/src/routes/acidentes.ts e backend/src/routes/doencas.ts, todas as rotas exigem autenticação, mas as rotas de criação não exigem perfil autorizado nem middleware de escopo. Nos respectivos controllers, a criação bloqueia apenas o perfil trabalhador e usa o trabalhadorId recebido no corpo, sem verificar que a pessoa-alvo pertence à empresa/unidade do usuário autenticado. Um usuário autenticado de perfil saude ou um gestor de outra empresa pode, portanto, tentar gravar registros para IDs fora do próprio escopo. O controller de trabalhador também precisa de revisão semelhante: a rota POST /api/trabalhadores exige autenticação, mas não usa middleware de escopo por empresa/unidade.

Há um padrão semelhante em submódulos do trabalhador: `backend/src/routes/submodulosTrabalhador.ts` protege as rotas com autenticação, e os controladores precisam validar o perfil e o vínculo entre o item e o trabalhador antes de criar, alterar ou excluir. Para servidores/funcionários, as rotas de escrita restringem os perfis a administrador/gestor, mas `trabalhadorId` pode ser fornecido ou alterado; é necessário garantir que esse trabalhador pertence à empresa do gestor, inclusive depois da atualização.

Mudança: definir uma matriz de permissões explícita para admin, gestor, saude e trabalhador, aplicar negação por padrão e validar o trabalhador-alvo em toda escrita e leitura sensível. O filtro de empresa precisa ser parte da própria consulta MongoDB; não basta carregar um objeto por `_id` e confiar apenas na interface. Em atualização, não permitir reatribuição de trabalhador sem nova validação. Adicionar testes de integração por perfil e por duas empresas: cada usuário tenta ler/criar/alterar/excluir registros do próprio escopo e do escopo alheio. A matriz de permissões é decisão de produto; até que esteja definida, o comportamento seguro é negar operações sem autorização explícita.

1.4.3 A2 — Helper de escopo pode falhar aberto para gestor sem empresa

Prioridade: alta, sob a condição de existir conta gestor sem empresa. Em `backend/src/utils/scope.ts`, `scopeFilterDirect` devolve `{}` para gestor quando `empresaScope` está vazio. Isso significa consulta sem filtro em vez de erro/resultado vazio para modelos diretos. O teste existente `authorization.test.ts` ainda valida esse comportamento permissivo, tornando-o um contrato de teste. A modelagem e os schemas permitem campos de empresa opcionais em alguns fluxos.

Mudança: se o token diz gestor e não existe uma empresa ativa, negar com 403 ou gerar um filtro impossível, nunca `{}`. Rever todos os consumidores de `scopeFilterDirect` e os middlewares de autorização. Alterar o teste para exigir falha fechada, incluindo dados legados e empresa/unidade inativas.

1.4.4 A3 — Access token válido por sete dias e segredo de refresh opcional

Prioridade: alta. `backend/render.yaml` define `JWT_EXPIRE: 7d`. Esse valor alimenta `generateToken` em `backend/src/utils/jwt.ts`; o frontend guarda o token de acesso em `sessionStorage` e o envia como `Authorization: Bearer` (`frontend/src/store/authStore.ts` e `frontend/src/services/api.ts`). Embora o cookie de acesso tenha vida configurada menor, o token bearer armazenado no navegador continua válido até o exp assinado: no deploy descrito pelo YAML, até sete dias. Além disso, `backend/src/config/config.ts` usa `JWT_SECRET` como fallback de `JWT_REFRESH_SECRET`, permitindo que os dois tipos de token compartilhem a mesma chave.

Mudança: configurar access token curto (o exemplo do projeto usa 15m) e refresh token separado, com segredo aleatório e independente; exigir ambos em produção em vez de usar fallback. Revisar a estratégia para tokens de acesso: preferir cookie `HttpOnly`, `Secure`, `SameSite` se compatível com o fluxo, ou manter bearer em memória e considerar que `sessionStorage` continua exposto a XSS. Adicionar testes de expiração, renovação, logout, revogação e separação entre tipos de token. A troca das chaves invalida sessões e deve ser planejada junto com o deploy.

Além disso, `backend/src/middleware/auth.ts` aceita JWT de acesso em query string. Tokens em URL podem acabar em histórico, logs de servidor/proxy e cabeçalhos `Referer`. Remova essa alternativa para JWT de sessão; não confunda esse token com links assinados de download, que devem permanecer restritos a um arquivo e com expiração curta.

1.4.5 A4 — Token de verificação de e-mail pode ser devolvido no cadastro

Prioridade: alta. `verificationToken` e `verificationTokenExpires` estão marcados `select: false` no modelo `User`, mas essa opção controla consultas ao MongoDB; ela não elimina automaticamente os campos de um documento recém-criado convertido com `toObject()`. O fluxo em `AuthService.register` converte o novo usuário para objeto e remove explicitamente a senha, enquanto `authController` devolve o objeto. Assim, o token que deveria provar posse do e-mail pode aparecer na resposta de cadastro, permitindo confirmar o próprio cadastro sem receber o e-mail. Isso anula a finalidade de verificação do endereço.

Mudança: a API deve retornar somente um DTO de usuário com campos permitidos — nunca `verificationToken`, `verificationTokenExpires`, tokens de refresh, hash/senha, códigos 2FA ou histórico de senhas. Remova explicitamente esses campos da resposta e adicione transformação segura de serialização. Teste que o retorno de cadastro não contém token e que a conta só é verificada pelo fluxo enviado ao endereço informado.

1.4.6 A5 — Uploads: Multer vulnerável, listagem sem filtro de empresa e armazenamento efêmero

Prioridade: alta. O backend usa `multer@1.4.5-lts.2`, em `diskStorage`. O aviso oficial do Express/OpenJS de 31/08/2026 recomenda Multer 2.3.0 ou superior: versões anteriores incluem falhas de DoS por nomes de campos multipart especialmente construídos, inclusive com possibilidade de crash do processo. A lista em `backend/src/routes/uploads.ts` permite a perfis `admin/gestor`, mas precisa limitar os metadados retornados ao escopo da empresa; o acesso individual ao arquivo já tem controles próprios, portanto liste metadados e download devem ser auditados separadamente.

O `render.yaml` declara plano `free` e o serviço grava arquivos localmente em `UPLOAD_DIR`. A documentação do Render informa que o filesystem de serviços gratuitos é efêmero; arquivos locais podem desaparecer em reinícios/deloys e não são armazenamento durável. Isso pode deixar referências MongoDB sem o arquivo correspondente.

Mudança: atualizar para a versão estável corrigida atual, no mínimo 2.3.0 conforme o aviso consultado, configurar os limites recomendados (`fieldArrayIndexLimit`, tamanho, quantidade, campos e timeout) e repetir testes de validação/MIME/assinatura/IDOR. Filtrar listagens e downloads pela empresa/trabalhador. Para persistência, escolher armazenamento de objeto privado (S3 compatível, por exemplo) ou uma opção de hosting com armazenamento adequado; não troque o provedor sem decisão de custo/contrato. Configurar backup, criptografia, retenção, nomes não previsíveis e links assinados curtos.

1.4.7 A6 — Health check declara pronto antes do banco estar conectado

Prioridade: média-alta. `server.ts` chama `app.listen()` antes de esperar o MongoDB. `connectDB()` captura a falha, agenda nova tentativa e não rejeita a promessa; o servidor pode continuar atendendo. O endpoint `/health` retorna 200 sem verificar a conexão do banco. Assim, o Render pode considerar o serviço saudável mesmo sem Atlas, e a resposta 200 observada nesta análise não prova disponibilidade funcional.

Mudança: separar `liveness` de `readiness`. A `readiness` deve retornar 503 enquanto MongoDB (e dependências indispensáveis) não estiver conectado; falha inicial deve impedir receber tráfego ou encerrar para que a plataforma reinicie. Aplicar `backoff` limitado e registrar eventos sem secrets. Aguardar conexão, índices/migrações e inicialização de jobs antes de marcar pronto; manter endpoints de `liveness` que não dependam do banco para permitir recuperação.

1.4.8 A7 — Logs de auditoria podem reter dados pessoais e não garantem cadeia íntegra

Prioridade: média-alta. `auditLogger` remove alguns nomes de campos de segredo, mas isso não equivale a mascarar CPF, endereço, e-mail, telefone, dados clínicos ou outros atributos de saúde. Há chamadas que passam objetos completos para auditoria. O middleware de leitura dispara registros sem aguardá-los; falhas são absorvidas para não bloquear a operação. O hash curto do modelo `AuditLog` não forma, por si só, uma cadeia encadeada que detecte alteração retroativa. A política/documentação do sistema precisa ser comparada com o prazo e o processo de retenção realmente implementados.

Mudança: registrar ação, entidade, identificador, campos alterados e resultado; evitar copiar documentos clínicos para detalhes; mascarar/remover PII recursivamente; usar lista permitida de campos; impedir logging de credenciais/tokens; estabelecer política de retenção e exclusão; proteger os logs com acesso restrito, backup e integridade/encadeamento se a rastreabilidade for requisito. Teste que CPF, token, endereço e relato clínico não aparecem em nenhum evento. Faça validação da política e dos prazos com o responsável de privacidade/jurídico; este relatório não é parecer jurídico.

1.4.9 A8 — Frontend sem type-check confiável e sem lint executável

Prioridade: alta para qualidade de entrega. O type-check encontrou 109 erros em 28 arquivos. Exemplos: tipos compartilhados não exportados (`IParametro`, `IQuestionario`, `IServidorFuncionario`), relações populadas tratadas como `never`, identificadores opcionais passados como obrigatórios, propriedade `lote` ausente em `IVacinacao`, e tipos de ícones `Lucide` mais restritos do que os componentes fornecidos. O arquivo `src/__tests__/masks.test.ts` também importa `@jest/globals`, que não está disponível no frontend. A build Vite ainda passa porque o script não executa `tsc`. O lint falha nos dois pacotes porque não há configuração ESLint.

Mudança: começar corrigindo os contratos em `frontend/src/types` (IDs/populates opcionais, formatos da API e módulos em falta), corrigir usos na UI sem espalhar `any`, decidir se os testes frontend terão runner/configuração próprios, e introduzir uma configuração ESLint compatível com a versão adotada. Depois, exigir type-check, testes, lint e build no CI e configurar Vercel para usar o mesmo gate. Não desative `strict` nem esconda erros com `casts` só para liberar deploy.

1.4.10 A9 — Dependências vulneráveis e CI possivelmente bloqueado

Prioridade: alta. A auditoria dos lockfiles mostrou advisories com correção disponível em diversas dependências. No recorte de produção do backend, os itens altos incluem `Axios`, `Joi` e `Nodemailer`; também aparecem avisos moderados em `Express/body-parser/qs`. No frontend aparecem `Axios`, `Vite`, `PostCSS` e `React Router` entre os pacotes diretos sinalizados, além de transitivas. A auditoria de `npm audit` enumera faixas afetadas, mas a explorabilidade depende de como cada pacote é usado. A sugestão automática para Vite aponta uma atualização de versão major; não é seguro aplicar `npm audit fix --force` sem migração e testes.

O CI usa Node 18, executa type-check e `npm audit --production`, e os jobs de deploy dependem desses gates. Os erros locais de tipos e os advisories atuais podem impedir novos deploys pelo workflow até serem resolvidos.

Mudança: atualizar primeiro os pacotes com risco de produção e patches compatíveis; tratar Multer em destaque; atualizar o lockfile e rodar testes/build de regressão. Para Vite/React Router ou outra alteração major, planejar migração em branch própria. Fixar a mesma versão suportada de Node em CI, Render e

Vercel, e testar a pipeline completa antes de mesclar. Não executar `npm audit fix --force` diretamente na branch de produção.

1.4.11 A10 — Configuração de produção depende de variáveis não declaradas no blueprint

Prioridade: média. `config.ts` usa `http://localhost:5173` como fallback para `FRONTEND_URL`, enquanto `backend/render.yaml` não documenta essa variável. Os caminhos de cadastro/recuperação usam Brevo, Resend ou SMTP; a chave de API e as credenciais não são declaradas no blueprint, portanto precisam estar configuradas manualmente no painel do Render. A presença no `.env` local não implica que a variável exista no serviço publicado. Sem URL correta, os links de verificação/reset podem apontar para localhost; sem provedor configurado, o fluxo de e-mail pode falhar ou cair em SMTP.

Mudança: conferir o painel Render e documentar nomes/necessidade (sem valores) para `FRONTEND_URL`, `BREVO_API_KEY` ou `RESEND_API_KEY`, `JWT_REFRESH_SECRET`, `MONGODB_URI` e demais integrações. Definir `FRONTEND_URL` como o endereço público correto; escolher apenas um provedor transacional validado; testar cadastro, verificação, recuperação de senha e 2FA. Exigir segredo de refresh separado e rejeitar config incompleta em produção.

1.4.12 A11 — Relatório público e retenção precisam de revisão de privacidade

Prioridade: média. Há endpoint público de relatório agregado. A supressão de grupos pequenos em algumas dimensões é positiva, mas confirme que todas as métricas de topo também aplicam limiar mínimo e que combinações de filtro não permitem inferir dados de uma pessoa/grupo pequeno. Também compare a política exibida pela interface com as rotinas efetivas de retenção, anonimização e exclusão: os TTLs de usuário não provam, por si só, que documentos médicos, uploads e logs sigam o mesmo prazo.

Mudança: definir limiar mínimo de publicação, agrupar categorias raras, revisar consultas combináveis e documentar as bases de retenção por coleção/arquivo. Validar textos, prazos, consentimento, canal de privacidade e procedimento de solicitação com o encarregado/responsável jurídico.

1.5 Plano de execução recomendado

Agora, antes de qualquer nova implantação: confirmar se os valores de `backend/.env` e o conteúdo de `admin.txt` são reais; se forem, revogar/rotacionar as credenciais. Em seguida, corrigir os fluxos de escrita/escopo e mudar o helper de empresa ausente para fail-closed. Não executar migração de storage, mudança de matriz de perfis ou rotação de JWT sem combinar impacto de acesso e deploy.

Primeira entrega de segurança: reduzir expiração do access token; separar o segredo de refresh; retirar token de verificação da resposta; atualizar Multer; limitar listagem de uploads; tornar readiness dependente do Atlas. Cobrir cada alteração com teste que reproduza o comportamento vulnerável antes e prove o bloqueio depois.

Próxima etapa de qualidade: corrigir os 109 erros do frontend, configurar lint/test runner, tratar advisories sem atualização major cega, e ajustar o CI para bloquear deploy se uma regra crítica falhar. Fazer deploy de homologação, revisar logs e só então liberar produção.

Operação e privacidade: migrar uploads para storage durável/privado após escolher provedor/custo; validar backups e restauração Atlas; revisar retenção, logs de auditoria e relatório público. Executar teste de recuperação a partir de backup e teste de autorização com dois tenants.

1.6 Checklist de aceitação

Tema	Evidência para encerrar
Segredos	Nenhum valor real no pacote/repositório; credenciais antigas invalidadas e rotação confirmada
Multitenancy	Testes de dois tenants provam que leitura/escrita/exportação/upload não atravessam empresa; gestor sem empresa recebe 403/resultado impossível
Autenticação	Access token curto; segredo de refresh independente; token de verificação nunca aparece em JSON; renovação e logout testados
Upload	Multer corrigido; limites multipart definidos; lista/download respeitam escopo; arquivo continua acessível após restart/deploy
Inicialização	Readiness 503 sem Atlas; conexão e tarefas iniciais aguardadas; health 200 só quando dependências críticas prontas
Qualidade	Frontend type-check, lint, testes e build passam; CI e Vercel usam gates coerentes; backend continua com todos os testes verdes
Dependências	npm audit sem advisories altos aplicáveis ou exceções documentadas com mitigação e responsável
Privacidade	Audit logs minimizados; limiar do relatório público validado; retenção e restauração documentadas

1.7 Fontes externas consultadas

- Render, [Deploy for Free](#) e [Persistent Disks](#): limitações do filesystem e disponibilidade de armazenamento persistente.
- Express/OpenJS, [August 2026 Security Releases](#): atualização do Multer 2.3.0 e advisories de multipart.
- GitHub Advisory Database, [GHSA-wc9g-mqfw-jrwm](#): versões afetadas e patch 2.3.0.
- [NVD CVE-2026-3520](#): advisory anterior do Multer consultado como referência adicional.

Conclusão: a base do projeto compila no backend e os testes existentes passam, mas o sistema precisa primeiro fechar autorização/escopo, segredos, token de acesso e uploads. Depois, a prioridade é restaurar um gate de qualidade real no frontend e limpar as dependências. A presença das URLs online não substitui esses controles nem comprova configuração segura do painel de produção.